Role of Data Mining and Machine Learning in Software Reusability

Rabia Qayyum
Military College of Signals
NUST
Islamabad, Pakistan
rabiaqayyum7@gmail.com

Joveria Rubaab
Military College of Signals
NUST
Islamabad, Pakistan
jrubab@live.com

Umer Riaz
Military College of Signals
NUST
Islamabad, Pakistan
umer.riaz@live.com

Dr. Fahim Arif
Military College of Signals
NUST
Islamabad, Pakistan
fahim@mcs.edu.pk

2021 International Conference on Innovative Computing (ICIC) | 978-1-6654-0091-6/21/$31.00 ©2021 IEEE | DOI: 10.1109/ICIC53490.2021.9693064

***Abstract*—Machine learning and data mining have opened new streams in research and software development. Likewise, integration of machine learning with software development life cycle has provided opportunities for well-organized and effective development. Software reusability is an important aspect of SDLC. Therefore, automation of software reusability plays a dynamic role in SDLC. It curtails the cost and effort required for development of a software product. This paper provides a taxonomical mapping of reuse metrics and corresponding machine learning techniques applicable. Artificial intelligence techniques i.e., neural networks and clustering are nominated in the paper to check their applicability for automation of software reusability. A model is created to identify best machine learning practice amongst the nominated techniques for automation of software reusability. Evaluation of model based on survey results identifies ANN: the best technique among neural networks and clustering.**

Keywords: Component based software engineering (CBSE), Artificial Intelligence (AI) Artificial Intelligence in software engineering application levels (AI-SEAL), clustering, classification, neural network

I. Introduction

Software development life cycle is a time-consuming process, which involves financial and human resource. However, software reusability increases productivity. It also enhances quality and maintainability of software product. It makes a software product cost effective. Software reusability integrates knowledge discovery and data mining techniques in software engineering processes to generate the field of Software Intelligence [8]. AI generates intelligent software, which can later be reused using automated software reusability techniques [9].

Reusability can be categorized into two major components: code components and non-code components [13]. Reusability can be further measured on various metrics. Software metrics for calculating the ease of reusability of software projects are vital for accomplishing "development by reuse" and "development for reuse" [9].

The reuse metrics could help in the development of reusability prediction models that could be applied by software developers. In order to collect information relating to the shared expenditure involved in developing a new version of an existing software or updating an existing software version by knowing the total code that can be reused. Literature provides a variety of algorithms used for reusability. However, metrics required for reuse are not identified explicitly in literature. Therefore, the paper identifies software metrics for object-oriented programming paradigm and procedure-oriented programming paradigm. Neural networks and clustering techniques are then nominated to check their usage in automation of software reusability. It then maps these metrics to neural network and clustering techniques to create a comprehensive taxonomy. Later paper uses proposed taxonomy to suggest a model for automation of software reusability. Evaluation of model is done using survey. The survey results identify ANN as best technique for software reusability.

The taxonomical mapping of data mining and machine learning techniques for a corresponding reuse metrics has been constructed by analysis of various research studies with respect to reuse metrics. The selection of papers for study is based on a criteria where only those papers are taken into consideration which have discussed software reusable metrics. Initially, the papers which discuss automating software reusability are selected. This abstract level of study phases out the software reuse metrics. The next level of study includes the papers which describe the techniques used for finding and implementing software reusability components. The final papers for study included the papers which discuss the techniques with respect to the metrics. The final taxonomy represents the metrics which become useful as a feature for a particular technique.

Therefore, in Section II, the studied literature has been discussed which explains the need of data mining and machine learning in the field of software engineering especially software reusability. In Section III, discusses the research conducted regarding the techniques of reusability. The subsection III-A, provides taxonomical mapping of data mining and machine learning techniques for a corresponding reuse metrics. Neural networks technique is used to identify and retrieve components that best fit the functionality of component for reuse. In subsection III-B, a model is suggested using classification, clustering and neural networks where identified taxonomy helps in building the basis of model. Later in the section, an evaluation of the model is provided based on the survey results.

II. Literature Review

We have reviewed a variety of papers for our research. In [8], a comprehensive taxonomical detail is provided of various AI techniques in software engineering. The taxonomy has been named as AI-SEAL. It guides investigators and experts to connect, understand the pros and cons of applying AI approaches for development of software. The taxonomy gives a guidance about which specific algorithm can be utilized for reusability. The three dimensions proposed as important are Type of AI (TAI) applied, Point of Application (PA), and Level of Automation (LA) offered. The paper further explains application of PA in SWEBOK's knowledge areas as to where and when AI can be applied. In [9], CBSE is described as emerging trend. The techniques such as K-mean and cosine similarity have been applied to various reuse non-code components i.e. project plan, architecture, design, detailed architecture & pattern, source code and test cases. The metrics for reuse are identified as weighted method per class (WMC), depth of inheritance tree (DIT), number of children (NOC), coupling between classes (CBO) and response for class (RFC). In paper [13] L. Kaur and A. Mishra have used classification models also known as Meta-classifiers to classify data based on seven reusable metrics on four version of same software. The metrics used for classification are coupling between objects (CBO), efferent coupling (EC), depth of inheritance (DIT), lack of cohesion between methods (LCOM), number of calls, number of methods and cyclomatic complexity (CC). The seven Meta classifiers used in paper are: adaBoost, filtered, M-class, bagging, random sub space, stacking and voting. In paper [9], component clustering method is used for grouping of component with similar characteristics. K-mean, K-mode, K-mediod and hierarchal clustering are the data mining approaches used in this paper. Once the cluster is designed, anticipated component can be found by searching the cluster. Hence, this technique proves beneficial for reuse of components. In paper [6], N.Krishna explains that CBSE divides the development in two aspects "Development for reuse" and "Development by reuse.". Therefore, massive repositories of components are created to be reused later. Neural network algorithms effectually scan the repositories to identify and retrieve specific component. The paper explains that reusability can be measured on two schemes i.e. empirical and qualitative.

In paper [21], D.Priyadarshni and Wangoo have recognized various AI techniques that lead to software intelligence. The paper explains that intelligent knowledge discovery can be performed using AI techniques on software engineering data to create software intelligence. Software intelligence leads to intelligent software reuse which finally generates intelligent automation. Various AI techniques are mapped to software process. Techniques included in the paper are artificial neural networks, neuro-fuzzy neural networks, evolutionary algorithms, fuzzy system etc. All the aforementioned techniques are fruitful for software reusability.

In [3], a case study about Microsoft has been discussed that Microsoft is currently using AI techniques in their pre-existing software development processes. Three challenges have been faced by developers when applying AI techniques to software development processes; Discover, organize, manage and version the data needed for machine learning applications is much more complicated and difficult than other domains of software development. Personalizing and customizing the model and to reuse a model requires different skillset by the software development team. AI components are more complex to tackle as seperate entities and modules than typical software modules— models may be "entangled" in a complicated nature of non-continous error behavior. The author has conducted surveys in form of interviews and observing software flows. The finding are presented in statistical form that identify each challenge and corresponding measures used for addressing it. Paper [4], uses software processes and maps it to state of art AI techniques. For example it maps knowledge based systems to requirement engineering as it identifies that reuse of experts design knowledge can play a substantial part in refining quality and productivity of the software development process. Similarly, other processes are linked to their effective AI methods.

III. Our Proposed framework

This section provides taxonomy for two techniques of data mining i.e. Clustering and Classification and one technique of machine learning i.e. neural network which is also used in combination for predicting and measuring the reusability later in the section. The taxonomy discusses the usage of techniques and their purpose. The section also presents a model suggesting classification, clustering and neural networks. Later in the section, an evaluation of the model is provided based on survey results.

A. Taxonomical Mapping

The key purpose of software reusability is to curtail recurrence of effort, time and cost. According to [17], the US department of defence solely could save 300 million dollars annually by reusing software components to increase reusability level as little as 1%.

At a high level, two aspects of reusability are found i.e. usability and usefulness [12].

Reusability = Usability + Usefulness

Usability is the "ease of use" of the component being used and is not dependent on the features of the component. Whereas, usefulness is suitability for the required purpose i.e. it depends on functionality, quality and generality of the components. Keeping this definition of reusability, steps [18] required for reusability are identified in Fig.1.

Multiple metrics are required for applying data mining and machine learning techniques for reusability. Table. I identifies metrics for object oriented paradigm [13],[18],[20],[23], whereas Table. II represents metrics for procedural paradigm [18].

Table. III and table. IV provides taxonomy for two techniques of data mining i.e. Clustering and classification and

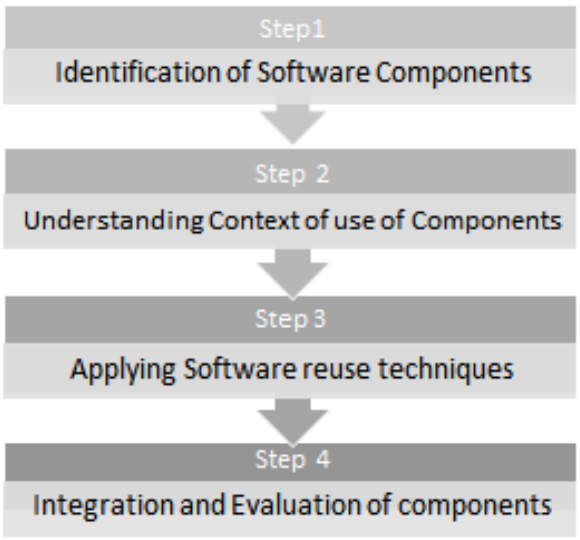

Fig. 1. Steps followed for applying reusability in sofware engineering

TABLE I
SOFTWARE REUSABILITY METRICS FOR OBJECT ORIENTED PROGRAMMING PARADIGM

Metrics
Coupling between classes (CCBC)
Number of children of a class (NOC)
Depth of Inheritance tree (DIT)
Weighted method per class (WWC)
Response for Class (RFC)
Method hiding factor (MHF)
Method Inheritance factor (MIF)
Polymorphism Factor (PF)
Efferent Coupling (EF)
Coupling between object classes (CBO)
Lack of cohesion in methods (LCOM)
Number of Interfaces
Class size
Number of Classes

one technique of machine learning i.e. neural network which is also used in amalgamation for forecasting and calculating the reusability [18], [16]. They are mapped to metrics, which are used for calculating reusability of an artifact using this technique.

TABLE II
SOFTWARE REUSABILITY METRICS FOR PROCEDURE ORIENTED PROGRAMMING PARADIGM

Metrics
Cyclomatic complexity (CC)
Cyclomatic density (CD)
Total lines of code (TLOC)
Comments lines of code (CLOC)
Executable lines of code (ELOC)
Blank lines of code (BLOC)
Node code (NC)
Halstead Metrics
Reuse Frequency Metrics
Regularity Metrics
Coupling Metrics

TABLE III
MAPPING OF SOFTWARE REUSABILITY METRICS WITH TECHNIQUES OF DATA MINING AND MACHINE LEARNING FOR OBJECT ORIENTED PROGRAMMING PARADIGM

Software Reusability Metrics	Techniques for Reusability
Coupling between classes (CCBC) [10]	• K-mean Clustering
Number of children of a class (NOC) [19]	• K-mean Clustering • Hierarchal Clustering • Neuro-Fuzzy Interface • Feed forward Neural Network • K- Mode clustering • Support Vector Machine Classifier • K-NN Classifier
Depth of Inheritance tree (DIT) [19]	• K-mean Clustering • Hierarchal Clustering • Feed forward Neural Network • K- Mode clustering • Support Vector Machine Classifier • K-NN Classifier • Meta Classifiers
Weighted method per class (WWC) [7]	• K-mean Clustering • Hierarchal Clustering • Feed forward Neural Network • K- Mode clustering • Support Vector Machine Classifier • K-NN Classifier • Meta Classifiers
Response for Class (RFC) [1]	• Meta Classifiers • Support Vector Machine Classifier • K-NN Classifier
Method hiding factor (MHF) [1]	• Hierarchical clustering
Method Inheritance factor (MIF) [1]	• K-NN Classifier
Polymorphism Factor (PF) [19]	
Efferent Coupling (EF) [19]	• Meta Classifiers
Coupling between object classes (CBO) [19]	• Hierarchal Clustering • Feed forward Neural Network • K- Mode clustering • Support Vector Machine Classifier • K-NN Classifier
Lack of cohesion in methods (LCOM) [19]	• K-mean Clustering • Hierarchal Clustering • Feed forward Neural Network • Support Vector Machine Classifier • Meta Classifiers
Number of Interfaces	
Class size	
Number of Classes	• Feed forward Neural Network

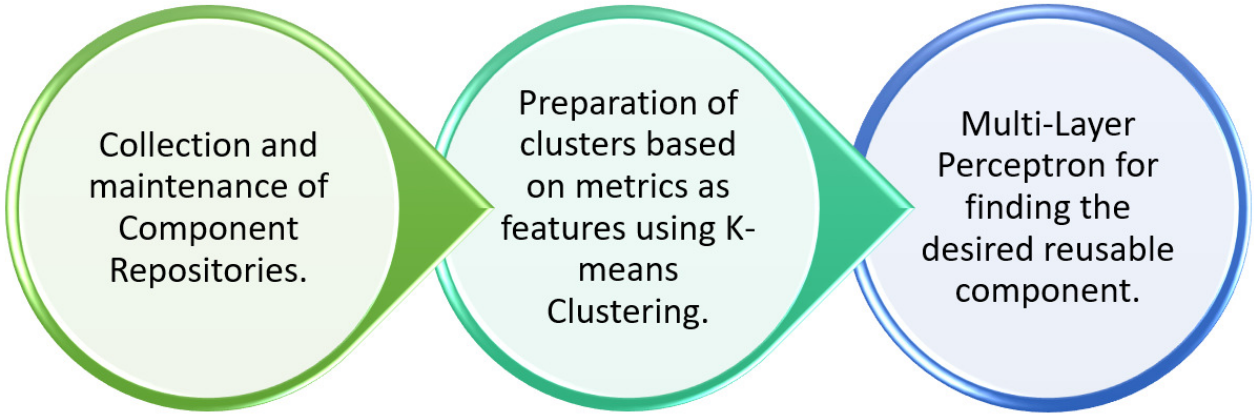

Fig. 2. Model to improve quality with respect to reusability achieved using Machine Learning and Data mining

B. Model to improve Quality

Clustering refers to grouping together components with same or similar characteristics in one cluster. It then makes it easier and faster for searching and retrieving components as the search space is reduced. In other words clustering makes "Divide and conquer" rule possible [13]. In paper [18] it is proposed that reusability of object oriented software can be predicted using clustering approach with software reusability metrics. [21] suggests that the clustering technique is not only useful in object oriented software but for procedure oriented software paradigm as well. In both cases, the main function for which clustering is used is predicting reusability of the components based on the concluded metrics. The similar software components are then combined in clusters. The metrics are important to generate Meta information about the software component which acts as the attribute when different clustering techniques and algorithms are applied. Therefore, the metrics mentioned in Table III and Table IV will be used to generate clusters per metrics.

Artificial neural networks replicate working pattern of human nervous system (working of brain). ANN is a varying deep learning technology originating from diverse domain of AI. Deep learning is a subpart of machine learning which includes neural networks. The motivation behind these approaches of neural networks is the way the human brain functions and thus scientists believe in going towards real AI. There are several types of neural networks algorithms i.e. Feed Forward- Artificial Neural Network, Radial Neural Network, Convolution Neural Network, Recurrent Neural Network, Modular Neural Network, etc. The performance of neural networks depends on the size of the dataset used, the bigger the dataset, better the performance is. Therefore, they can be effectively used in software reusability.

The model proposed in the paper will use K-means clustering [14] on component repository, which clusters the data with highest similarity within a cluster and lowest similarity within different clusters are using K-means. The data is partitioned into different levels of reusability value based on the features and attributes. K-means produces K number of clusters. For each cluster there is a centroid and each data value is related to the closest centroid with highest similarity [13]. K-means clustering is fast and most efficient hard clustering techniques producing tight clusters when the number of attributes or values increases [14].

In the first step of constructing model, clusters will be created using metrics identified in taxonomy for OO Paradigm are as Number of Children, Lack of Cohesion in Methods, Weighted Methods per Class, Coupling Between Object Classes and Depth of Inheritance Tree, since all the five metrics are commonly used by both K-mean clustering and ANN. Once the clusters are formed, single hidden perceptron will be used to identify desired component from gathered clusters. One by one, ANN will work on all clusters and will give 0 OR 1 input, corresponding to found or not found.

Figure. 2 identifies the model suggested in the paper. Accuracy of model can be tested on a data set once it is implemented in practice. Clusters with gather similar components together and ANN will take less time to identify desired component.

IV. Survey Analysis

Scholars and intellectuals of the domain helped in model evaluation. The questionnaire covered three important aspects i.e automation of of reusability, effectiveness of machine learning in reusability and the design of the model. Google forms was the medium used to conduct this survey. The domain experts who participated in the survey work in software houses with minimum two year work experience. A two weeks time period was allocated to collect the survey results. The gathered data and analysis is presented below.

Fig. 3 displayed response regarding automating reusability for survey question, where helpfulness of automating reusability in the field of software industry was questioned. 89.7% people responded in favour of the idea automating reusability. Fig. 4 shows the response for software repositories that can be used for finding reusable software component which tends

TABLE IV
MAPPING OF SOFTWARE REUSABILITY METRICS WITH TECHNIQUES OF DATA MINING AND MACHINE LEARNING FOR PROCEDURE ORIENTED PROGRAMMING PARADIGM

Software Reusability Metrics	Techniques for Reusability
Cyclomatic complexity (CC) [19]	• K-mean Clustering • Hierarchal Clustering • Feed forward Neural Network • K- Mode clustering • Support Vector Machine Classifier • K-NN Classifier
Cyclomatic density (CD) [5]	
Total lines of code (TLOC)	• K-NN Classifier
Comments lines of code (CLOC) [15]	• K-NN Classifier
Executable lines of code (ELOC) [15]	• K-mean Clustering • K- Mode clustering • K-NN Classifier
Blank lines of code (BLOC) [22]	
Node code (NC) [2]	
Halstead Metrics [22]	• Feed forward Neural Network • Support Vector Machine Classifier
Reuse Frequency Metrics	• Feed forward Neural Network • Support Vector Machine Classifier
Regularity Metrics [2]	• Feed forward Neural Network • Support Vector Machine Classifier
Coupling Metrics [11]	• Feed forward Neural Network • Support Vector Machine Classifier

to collect data regarding availability of software components repositories. 51.7% response rate displays that there are sufficient software repositories while 17.2% disagree with their availability.

Fig. 5 depicts that how effective clustering can be to be find similar component in a large repository of components. 55.2% people responded that clustering is highly effective. Fig. 6 displays response percentage for K-mean clustering suggested as highly effective algorithm for large datasets. While 37.9% percent people believe K-mean clustering is highly effective, 51.7% still responded with "Maybe". This shows that K-means clustering has yet to be tested on multiple large repositories as many people are unaware of its effectiveness. Fig. 7 displays response of people regarding use of Artificial neural networks for finding a specific attribute in a large pool of datasets. 89.7%

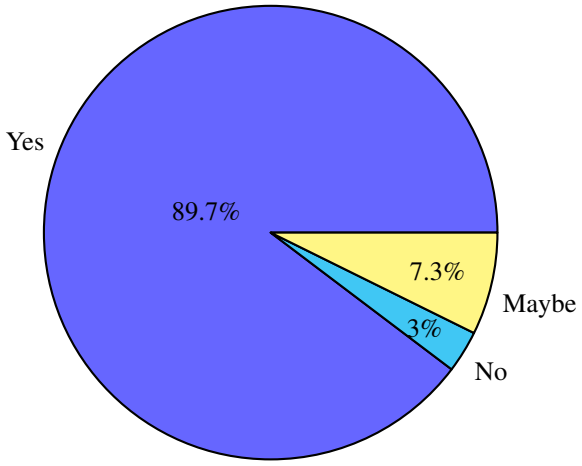

Fig. 3. Automating Reusability

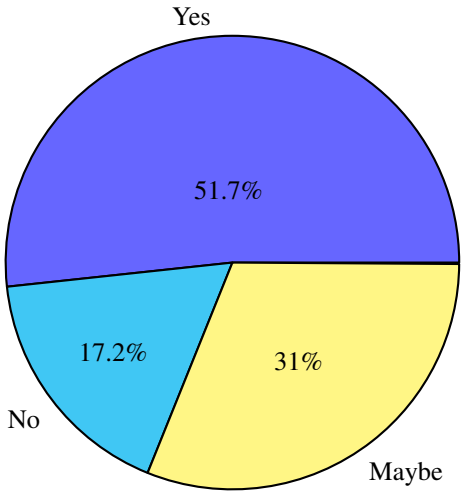

Fig. 4. Availablilty repositories for finding reusable components

people think ANN is highly effective in automating reusability.

Fig. 8 displays response of people regarding effectiveness of using combined algorithms K-mean and artificial neural network in automating reusability, 69% people think ANN when used in combination with K-mean clustering can produce effective results. Fig. 9 displays response of people regarding effect of automating reusability on cost of the product, where people are asked if artificial intelligence on reusability is cost effective decision. For this, 68.6% people believe it can be highly cost effective and 34.5% responses state "Maybe" and only 6.9% response rate favours that it is not a cost effective process. Fig. 10 displays response of people regarding feasibility of the model where 72.4% believe it is feasible to implement on data repositories and 27.6% responses state "Maybe".

As discussed that clustering gathers similar data at one place while, neural networks identify component from a library. Therefore, aforementioned technique works best when used together to achieve a certain goal of finding and identifying correct component in the case of "development by reuse".

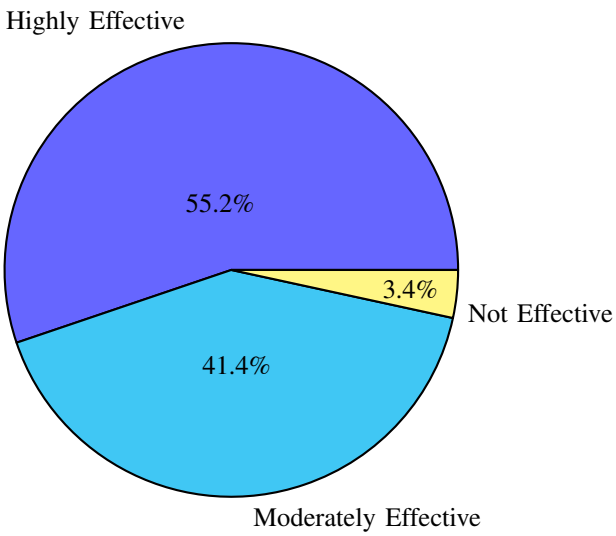

Fig. 5. Effectiveness of clustering in reusability

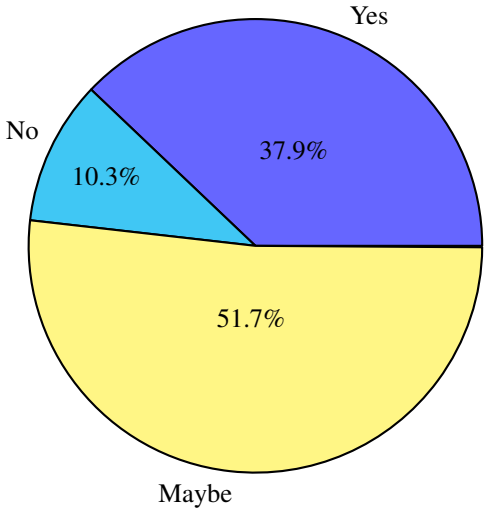

Fig. 6. Effectiveness of K-mean clustering

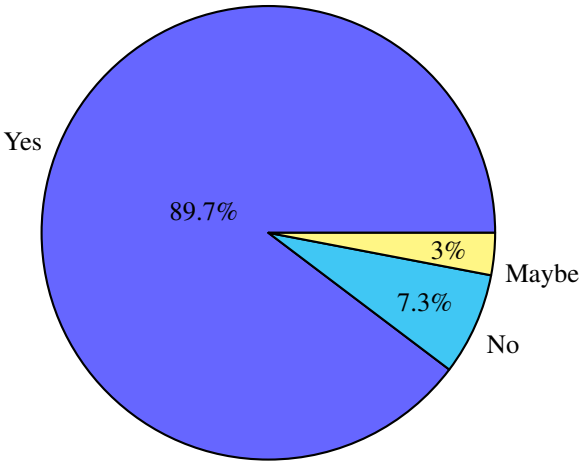

Fig. 7. Effectiveness of ANN

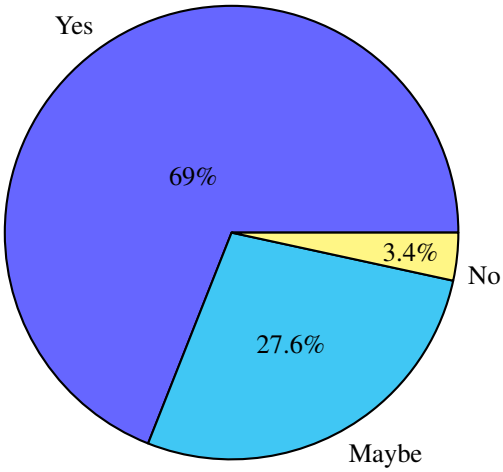

Fig. 8. Effectiveness of combined ANN and K-mean

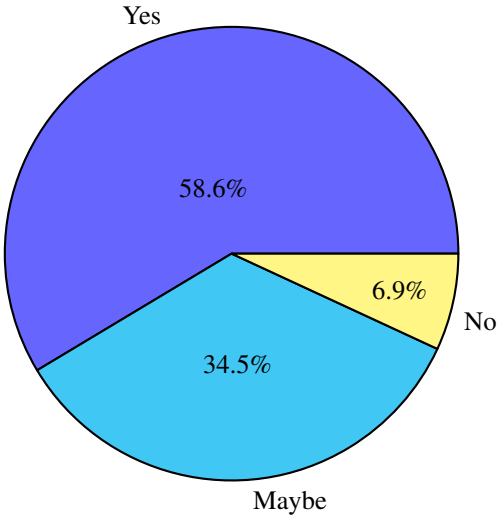

Fig. 9. ANN and cost effecctiveness

The research suggests there is no best technique amongst the three, rather they support each other in reuse. Evaluating our model, it turns out that it curtails the effort, time and cost in identifying and reusing the components. Thus practices of data mining and machine learning gives best results when used in combination. The model's accuracy can be improved and enhanced over the period of time.

V. Conclusion

Software development process is enhanced by software reusability as it saves time, budget and effort for a developer to build same general functionality again. Software reusability is a cost effect technique that not only helps the developer but also the product achieved is of high quality. Reusability of a software component is measure on various metrics. These metrics help data mining and machine learning techniques to help achieve and use reusable software components. The taxonomical mapping explains the neural network and data mining techniques and their respective metrics that help in measuring reusability. The second part of the paper proposes

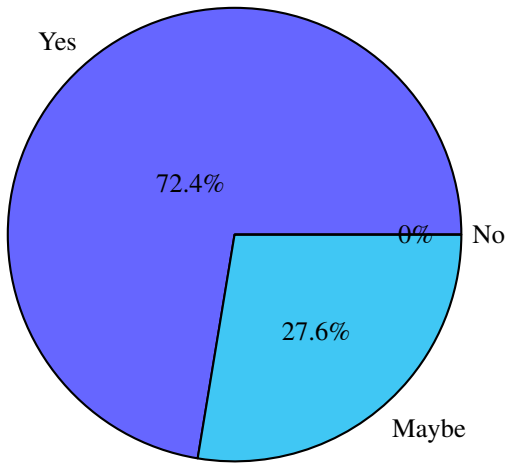

Fig. 10. Correctness of model

a model based on k-means clustering and feed-forward neural network which would help developers finding the reusable components from the huge repositories maintained. The survey is conducted to evaluate the efficiency of the proposed model according to the people working in software industry. From the survey results, it is concluded that the model proposed will be helpful in the development of software using reusable components.

VI. Future Work

The paper proposed a model which suggests Machine learning and data mining techniques to be used for finding reusable components. The future work is planned to implement and deploy this model to gather experience from software industry and enhancing model based on the information gathered.

References

[1] KK Aggarwal et al. "Software reuse metrics for object-oriented systems". In: *Third ACIS Int'l Conference on Software Engineering Research, Management and Applications (SERA'05)*. IEEE. 2005, pp. 48–54.

[2] Jehad Al Dallal. "Software similarity-based functional cohesion metric". In: *IET software* 3.1 (2009), pp. 46–57.

[3] Saleema Amershi et al. "Software engineering for machine learning: a case study". In: *Proceedings of the 41st International Conference on Software Engineering: Software Engineering in Practice*. IEEE Press. 2019, pp. 291–300.

[4] Hany H Ammar, Walid Abdelmoez, and Mohamed Salah Hamdi. "Software engineering using artificial intelligence techniques: Current state and open problems". In: *Proceedings of the First Taibah University International Conference on Computing and Information Technology (ICCIT 2012), Al-Madinah Al-Munawwarah, Saudi Arabia*. 2012, p. 52.

[5] Marcus AS Boxall and Saeed Araban. "Interface metrics for reusability analysis of components". In: *2004 Australian Software Engineering Conference. Proceedings.* IEEE. 2004, pp. 40–51.

[6] N Krishna Chythanya and Lakshmi Rajamani. "Neural Network Approach for Reusable Component Handling". In: *2017 IEEE 7th International Advance Computing Conference (IACC)*. IEEE. 2017, pp. 75–79.

[7] Fatma Dandashi. "A method for assessing the reusability of object-oriented code using a validated set of automated measurements". In: *Proceedings of the 2002 ACM symposium on Applied computing*. 2002, pp. 997–1003.

[8] Robert Feldt, Francisco G de Oliveira Neto, and Richard Torkar. "Ways of applying artificial intelligence in software engineering". In: *Proceedings of the 6th International Workshop on Realizing Artificial Intelligence Synergies in Software Engineering*. ACM. 2018, pp. 35–41.

[9] "[Front matter]". In: *2016 3rd International Conference on Computing for Sustainable Global Development (INDIACom)*. 2016, pp. i–64.

[10] Gui Gui and Paul D. Scott. "Measuring Software Component Reusability by Coupling and Cohesion Metrics". In: *J. Comput.* 4 (2009), pp. 797–805.

[11] Gui Gui and Paul D Scott. "Measuring Software Component Reusability by Coupling and Cohesion Metrics." In: *J. Comput.* 4.9 (2009), pp. 797–805.

[12] Capers Jones. *Estimating software costs: Bringing realism to estimating*. McGraw-Hill Companies New York, 2007.

[13] Loveleen Kaur and Ashutosh Mishra. "An Empirical Analysis for Predicting Source Code File Reusability Using Meta-Classification Algorithms". In: *Advanced Computational and Communication Paradigms*. Springer, 2018, pp. 493–504.

[14] Ajay Kumar. "Measuring Software reusability using SVM based classifier approach". In: *International Journal of Information Technology and Knowledge Management* 5.1 (2012), pp. 205–209.

[15] Sonia Manhas et al. "Identification of reusable software modules in function oriented software systems using neural network based technique". In: *World Academy of Science, Engineering and Technology* 67 (2010), pp. 823–827.

[16] Marko Mijač and Zlatko Stapić. "Reusability metrics of software components: survey". In: *26th Central European Conference on Information and Intelligent Systems (CECIIS 2015)*. 2015.

[17] Jeffrey S Poulin. *Measuring software reuse: principles, practices, and economic models*. Addison-Wesley Reading, MA, 1997.

[18] BV Ajay Prakash, DV Ashoka, and VN Manjunath Aradhya. "Application of data mining techniques for software reuse process". In: *Procedia Technology* 4 (2012), pp. 384–389.

[19] Parvinder Singh Sandhu and Hardeep Singh. “A reusability evaluation model for OO-based software components”. In: *International Journal of Computer Science* 1.4 (2006), pp. 259–264.

[20] Anju Shri et al. “Prediction of reusability of object oriented software systems using clustering approach”. In: *World academy of science, Engineering and Technology* 43 (2010), pp. 853–856.

[21] Divanshi Priyadarshni Wangoo. “Artificial Intelligence Techniques in Software Engineering for Automated Software Reuse and Design”. In: *2018 4th International Conference on Computing Communication and Automation (ICCCA)*. IEEE. 2018, pp. 1–4.

[22] Hironori Washizaki, Hirokazu Yamamoto, and Yoshiaki Fukazawa. “A metrics suite for measuring reusability of software components”. In: *Proceedings. 5th International Workshop on Enterprise Networking and Computing in Healthcare Industry (IEEE Cat. No. 03EX717)*. IEEE. 2004, pp. 211–223.

[23] Syeda Iffat Zahara, Muhammad Ilyas, and Tehseen Zia. “A study of comparative analysis of regression algorithms for reusability evaluation of object oriented based software components”. In: *2013 International Conference on Open Source Systems and Technologies*. IEEE. 2013, pp. 75–80.